

SUMMARY

This topic covers a comprehensive analysis of Stack data structures, their implementation, operations, applications, stack permutations, and expression evaluation using stacks. It emphasizes the distinction between abstract and concrete views, the LIFO principle, array-based implementation mechanics, and stack-based algorithms for expression conversion. Mastery of these concepts is crucial for both competitive programming and system-level programming tasks.

Stacks are linear data structures operating under the **Last-In-First-Out (LIFO)** principle. Key points include:

- **Fundamental Policy:** Elements are inserted and removed from the same end, the Top of the Stack.
- **Operational Efficiency:** Push and Pop operations exhibit **$O(1)$** time complexity.
- **Implementation Mechanics:** Array-based stacks manage **Overflow** and **Underflow** conditions via the Top index.
- **Applications:** Stacks are essential for **system function call management, expression conversion, undo operations**, recursion, and algorithmic tasks like **DFS** and **Tower of Hanoi**.
- **Stack Permutations:** Only certain output sequences are possible due to LIFO constraints.
- **Expression Evaluation:** Postfix and prefix notations enable single-pass computation using stacks.

1. DEFINING DATA STRUCTURES: ABSTRACT VS. CONCRETE VIEWS

1.1 Abstract View (Abstract Data Type - ADT)

- Focuses on **conceptual definition** without involving code.
- Defines **operations** and features rather than implementation.
- Real-world analogy: A stack of plates, bangles, or CDs, where items are added and removed from the same end.

1.2 Concrete View

- Focuses on **actual implementation**, coding, and memory management.
- Handles **physical constraints**, such as fixed array sizes and memory availability.

2. THE STACK DATA STRUCTURE: FUNDAMENTALS

2.1 The LIFO Policy

- **Last-In-First-Out (LIFO)** or **First-In-Last-Out (FILO)** governs stack behavior.
- **Single End Access:** Insertions and deletions occur only at the **Top of Stack**.

2.2 Standard Operations

Operation	Description
Push	Insert an element onto the stack.
Pop	Delete/remove the top-most element.
isEmpty()	Returns True if the stack is empty, False otherwise.
Top	Tracks the most recently added element (variable, index, or pointer).

3. CONCRETE IMPLEMENTATION VIA ARRAYS

3.1 Initialization and Indices

- Pre-allocate stack size (e.g., `Max_Size = 10`).
- Initialize **Top = -1** for an empty stack.
- **Top** tracks the index of the last inserted element.

3.2 Error Conditions

- **Overflow:** Push on a full stack (`Top == Size - 1`).
- **Underflow:** Pop from an empty stack (`Top == -1`).

3.3 Complexity and Performance

- Push and Pop involve simple index increments/decrements and direct memory access.
- Both operations have **O(1) time complexity**.

4. STACK APPLICATIONS AND THE "WAITING" PRINCIPLE

4.1 System Stack and Function Calling

- Manages execution order for nested function calls.
- **Activation Records:** Each function call is saved on the system stack.
- **LIFO Return:** Functions resume in reverse order of calls once the current function completes.

4.2 Additional Applications

- Undo operations in software.
- Recursion management.
- Parenthesis checking in expressions.
- Depth-First Search (DFS) in graphs.
- Expression conversion: Infix $\leftrightarrow$ Postfix $\leftrightarrow$ Prefix.

- Solving Tower of Hanoi.

5. STACK PERMUTATIONS

- Permutations must respect the **fixed insertion order**.
- Pop operations can occur at any point but the LIFO policy restricts possible sequences.
- **Example:** Input 1, 2, 3 $\rightarrow$ output 3, 1, 2 is invalid.

6. EXPRESSION NOTATIONS AND EVALUATION

- **Infix:** Operators between operands (e.g., $A + B$).
- **Postfix (Reverse Polish):** Operators after operands (e.g., $AB+$). Preferred for computer evaluation.
- **Prefix (Polish):** Operators before operands (e.g., $+AB$).

6.1 Infix to Postfix Conversion (Stack-Based Algorithm)

Token	Action
Operand	Add directly to output.
Left Parenthesis (	Push onto stack.
Right Parenthesis)	Pop from stack to output until matching '(' is found; discard both parentheses.
Operator	1. If stack empty or Top = '(', push. 2. If incoming operator > Top operator, push. 3. If incoming operator $\leq$ Top, pop Top to output and repeat check.
End of String	Pop all remaining operators to output.

6.2 Precedence and Associativity

- Power/Exponential: Highest, Right-to-Left associativity.
- Multiplication/Division: High, Left-to-Right associativity.
- Addition/Subtraction: Lowest, Left-to-Right associativity.